



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Кафедра математического и компьютерного моделирования

Избосарова Жасмина Олимжоновна

**ЛАБОРАТОРНАЯ РАБОТА №3: ЭМПИРИЧЕСКАЯ ФУНКЦИЯ
РАСПРЕДЕЛЕНИЯ**

Направление подготовки 02.03.01 СЦТ «Сквозные цифровые технологии»,
образовательная программа «Математика и компьютерные науки»,
очная форма обучения

Студент группы Б9123-02.03.01сцт2

_____ Ж. О. Избосарова

Руководитель проекта

_____ К. А. Ганжа

Владивосток

2026

1 Краткое математическое описание методов и синтаксис в Python

Эмпирическая функция распределения определяется формулой

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n I(X_i \leq x),$$

Доверительный интервал $L(x), U(x)$ для $F(x)$ с уровнем доверия $1 - \alpha$ в нашем случае имеет вид:

$$P(L(x) \leq F(x) \leq U(x)) \geq 1 - \alpha.$$

Где:

$$L(x) = \max(\hat{F}_n(x) - \varepsilon_n; 0),$$

$$U(x) = \min(\hat{F}_n(x) + \varepsilon_n; 1),$$

$$\varepsilon_n = \sqrt{\frac{1}{2n} \ln \left(\frac{2}{\alpha} \right)}.$$

$$\alpha = 0.05.$$

Равномерное распределение: $X \sim U(2, 7)$.

Код:

```
1 st.uniform.rvs(loc=2, scale=5, size=100, random_state=16)
2 st.uniform.rvs(loc=2, scale=5, size=1000, random_state=16)
```

Распределение Бернулли: $X \sim B(0.27)$.

Код:

```
1 st.bernoulli.rvs(p=0.27, size=100, random_state=16)
2 st.bernoulli.rvs(p=0.27, size=1000, random_state=16)
```

Биномиальное распределение: $X \sim Bin(12, 0.35)$.

Код:

```
1 st.binom.rvs(n=12, p=0.35, size=100, random_state=16)
2 st.binom.rvs(n=12, p=0.35, size=1000, random_state=16)
```

Нормальное распределение: $X \sim N(4.5, 1.8^2)$.

Код:

```
1 st.norm.rvs(loc=4.5, scale=1.8, size=100, random_state=16)
2 st.norm.rvs(loc=4.5, scale=1.8, size=1000, random_state=16)
```

Собственная функция вычисления ЭФР и доверительного интервала имеет вид:

```
1 def empirical_cdf(sample, x, alpha=0.05):
2     sample = np.sort(np.asarray(sample))
3     n = sample.size
4     x_array = np.atleast_1d(x)
5
6     fn = np.searchsorted(sample, x_array, side="right") / n
7     epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))
8     lower = np.clip(fn - epsilon, 0.0, 1.0)
9     upper = np.clip(fn + epsilon, 0.0, 1.0)
10
11     if np.ndim(x) == 0:
12         return float(fn[0]), float(lower[0]), float(upper[0])
13     return fn, lower, upper
```

Для сравнения используется встроенная функция из SciPy:

Код:

```
1 ecdf_result = st.ecdf(sample)
2 ecdf_values = ecdf_result.cdf.evaluate(x_grid)
3 ci = ecdf_result.cdf.confidence_interval(confidence_level
4     =0.95)
5 low = ci.low.evaluate(x_grid)
6 high = ci.high.evaluate(x_grid)
```

Для непрерывных распределений графики строятся на плотной равномерной сетке по x , а для дискретных по целочисленным значениям из области возможных исходов.

2 Результаты

2.1 Таблица результатов

Таблица 1: Значения ЭФР в точке медианы выборки и 95%-й доверительный интервал

Выборка	x	$F_n(x)$	Нижняя граница	Верхняя граница
uniform_100	4.6254	0.5000	0.3642	0.6358
uniform_1000	4.4807	0.5000	0.4571	0.5429
bernoulli_100	0.0000	0.7500	0.6142	0.8858
bernoulli_1000	0.0000	0.7500	0.7071	0.7929
binom_100	4.0000	0.5800	0.4442	0.7158
binom_1000	4.0000	0.5950	0.5521	0.6379
norm_100	4.6516	0.5000	0.3642	0.6358
norm_1000	4.4144	0.5000	0.4571	0.5429

2.2 Графики

На графиках:

- красная линия — истинная функция распределения;
- синяя ступенчатая линия — эмпирическая функция распределения;
- зелёные пунктирные линии — нижняя и верхняя границы 95%-го доверительного интервала.

На первом рисунке приведены результаты для собственной реализации ЭФР и доверительного интервала построенного по приведённой выше формуле

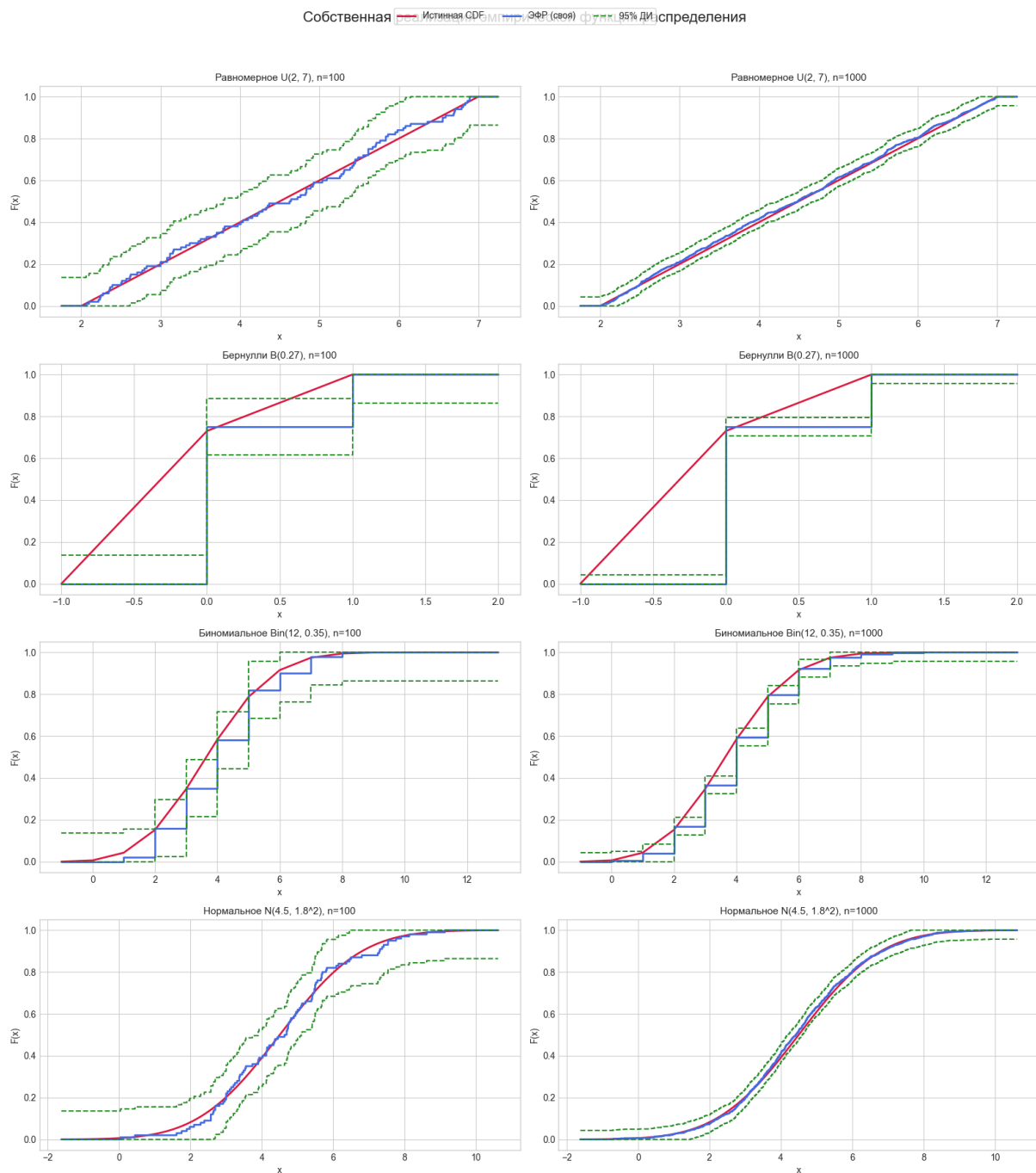


Рис. 1: Собственная реализация ЭФР и 95%-го доверительного интервала

На следующем рисунке приведены результаты, полученные с помощью `scipy.stats.ecdf`

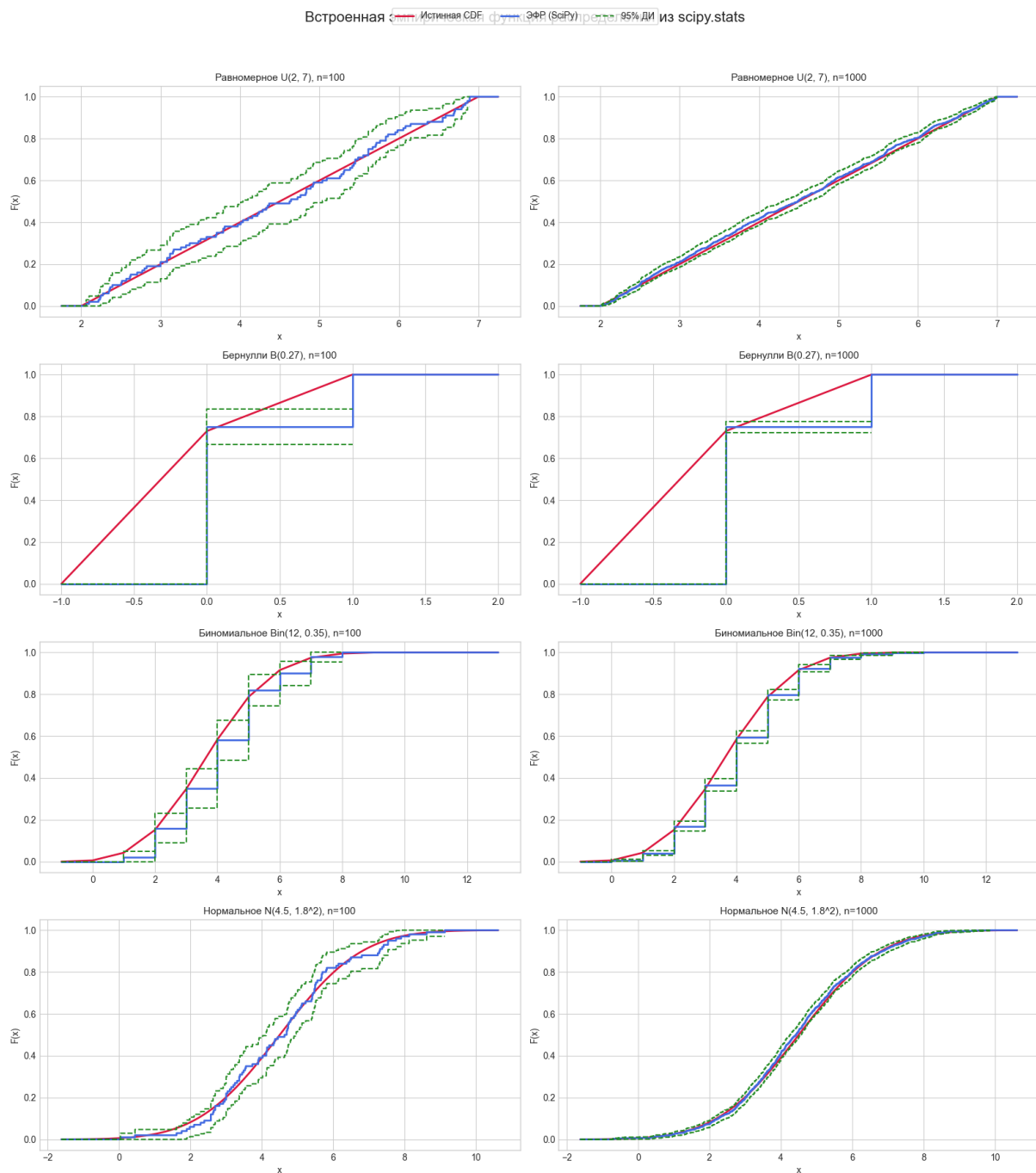


Рис. 2: ЭФР и 95%-й доверительный интервал с использованием `scipy.stats.ecdf`

3 Полный код программы

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import scipy.stats as st

```

```

5
6
7 plt.style.use("seaborn-v0_8-whitegrid")
8 plt.rcParams["figure.figsize"] = (16, 10)
9 plt.rcParams["axes.titlesize"] = 11
10 plt.rcParams["axes.labelsize"] = 10
11
12
13 random_state = 16
14
15
16 samples = {
17     "uniform_100": st.uniform.rvs(loc=2, scale=5, size=100,
18     random_state=random_state),
19     "uniform_1000": st.uniform.rvs(
20         loc=2, scale=5, size=1000, random_state=random_state
21     ),
22     "bernoulli_100": st.bernoulli.rvs(p=0.27, size=100,
23     random_state=random_state),
24     "bernoulli_1000": st.bernoulli.rvs(
25         p=0.27, size=1000, random_state=random_state
26     ),
27     "binom_100": st.binom.rvs(n=12, p=0.35, size=100,
28     random_state=random_state),
29     "binom_1000": st.binom.rvs(
30         n=12, p=0.35, size=1000, random_state=random_state
31     ),
32     "norm_100": st.norm.rvs(loc=4.5, scale=1.8, size=100,
33     random_state=random_state),
34     "norm_1000": st.norm.rvs(
35         loc=4.5, scale=1.8, size=1000, random_state=
36     random_state
37     ),
38 }
39
40 plot_config = [
41     {
42         "sample_name": "uniform_100",
43         "title": "Равномерное U(2, 7), n=100",

```



```

40     "kind": "continuous",
41     "dist": st.uniform(loc=2, scale=5),
42 },
43 {
44     "sample_name": "uniform_1000",
45     "title": "Равномерное U(2, 7), n=1000",
46     "kind": "continuous",
47     "dist": st.uniform(loc=2, scale=5),
48 },
49 {
50     "sample_name": "bernoulli_100",
51     "title": "Бернулли B(0.27), n=100",
52     "kind": "discrete",
53     "dist": st.bernoulli(p=0.27),
54 },
55 {
56     "sample_name": "bernoulli_1000",
57     "title": "Бернулли B(0.27), n=1000",
58     "kind": "discrete",
59     "dist": st.bernoulli(p=0.27),
60 },
61 {
62     "sample_name": "binom_100",
63     "title": "Биномиальное Bin(12, 0.35), n=100",
64     "kind": "discrete",
65     "dist": st.binom(n=12, p=0.35),
66 },
67 {
68     "sample_name": "binom_1000",
69     "title": "Биномиальное Bin(12, 0.35), n=1000",
70     "kind": "discrete",
71     "dist": st.binom(n=12, p=0.35),
72 },
73 {
74     "sample_name": "norm_100",
75     "title": "Нормальное N(4.5, 1.8^2), n=100",
76     "kind": "continuous",
77     "dist": st.norm(loc=4.5, scale=1.8),
78 },
79 {

```

```

80     "sample_name": "norm_1000",
81     "title": "Нормальное  $N(4.5, 1.8^2)$ ,  $n=1000$ ",
82     "kind": "continuous",
83     "dist": st.norm(loc=4.5, scale=1.8),
84 },
85 ]
86
87
88 def empirical_cdf(sample, x, alpha=0.05):
89     # преобразуем выборку к numpy-массиву и сортируем,
90     # чтобы затем быстро считать долю элементов  $\leq x$ .
91     sample = np.sort(np.asarray(sample))
92     n = sample.size
93     x_array = np.atleast_1d(x)
94
95     # значение эфр сколько элементов выборки не превосходят x, делим на
96     # n.
97     fn = np.searchsorted(sample, x_array, side="right") / n
98
99     # считаем эппсilon по формуле доверительного интервала
100    # и ограничиваем границы отрезком [0, 1].
101    epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))
102    lower = np.clip(fn - epsilon, 0.0, 1.0)
103    upper = np.clip(fn + epsilon, 0.0, 1.0)
104
105    # Если на вход подано одно число x, возвращаем обычные float,
106    # а не массивы из одного элемента.
107    if np.ndim(x) == 0:
108        return float(fn[0]), float(lower[0]), float(upper[0])
109    return fn, lower, upper
110
111 def make_x_grid(sample, kind, frozen_dist):
112     # строим сетку по x, на которой будем рисовать графики для
113     # непрерывных распределений берем плотную сетку,
114     # для дискретных - целочисленные значения.
115     sample = np.asarray(sample)
116     left, right = frozen_dist.support()
117
118     if kind == "continuous":
119         if np.isfinite(left) and np.isfinite(right):

```

```

119         span = right - left
120         pad = 0.05 * span if span > 0 else 1.0
121         return np.linspace(left - pad, right + pad, 600)
122
123     q_left = frozen_dist.ppf(0.001)
124     q_right = frozen_dist.ppf(0.999)
125     left = min(sample.min(), q_left)
126     right = max(sample.max(), q_right)
127     pad = 0.05 * (right - left) if right > left else 1.0
128     return np.linspace(left - pad, right + pad, 600)
129
130     left = int(np.floor(left)) - 1
131     right = int(np.ceil(right)) + 1
132     return np.arange(left, right + 1)
133
134
135 def build_demo_table(config):
136     # собираем таблицу в точке медианы каждой выборки
137     # считаем значение ЭФР и границы доверительного интервала.
138     rows = []
139     for item in config:
140         sample = samples[item["sample_name"]]
141         x0 = float(np.median(sample))
142         fn, low, high = empirical_cdf(sample, x0)
143         rows.append(
144             {
145                 "Выборка": item["sample_name"],
146                 "x": round(x0, 4),
147                 "F_n(x)": round(fn, 4),
148                 "Нижняя граница 95% ДИ": round(low, 4),
149                 "Верхняя граница 95% ДИ": round(high, 4),
150             }
151         )
152     return pd.DataFrame(rows)
153
154
155 def plot_with_custom_ecdf(ax, sample, title, kind,
156                             frozen_dist):
157     # рисуем на одном графике ,
158     # собственную ЭФР и доверительный интервал.

```

```

158     x_grid = make_x_grid(sample, kind, frozen_dist)
159     true_cdf = frozen_dist.cdf(x_grid)
160     emp_cdf, low, high = empirical_cdf(sample, x_grid)
161
162     ax.plot(x_grid, true_cdf, color="crimson", linewidth=2,
163 label="Истинная CDF")
164     ax.step(x_grid, emp_cdf, where="post", color="royalblue",
165 linewidth=2, label="ЭФР")
166     ax.step(
167         x_grid,
168         low,
169         where="post",
170         color="forestgreen",
171         linestyle="--",
172         linewidth=1.5,
173         label="95% ДИ",
174     )
175     ax.step(
176         x_grid,
177         high,
178         where="post",
179         color="forestgreen",
180         linestyle="--",
181         linewidth=1.5,
182     )
183
184     ax.set_title(title)
185     ax.set_xlabel("x")
186     ax.set_ylabel("F(x)")
187     ax.set_ylim(-0.05, 1.05)
188
189 def plot_with_scipy_ecdf(ax, sample, title, kind, frozen_dist
190 ):
191     # для сравнения строим график через готовую реализацию
192     # берем ЭФР, а также ее доверительный интервал встроенными методами.
193     x_grid = make_x_grid(sample, kind, frozen_dist)
194     true_cdf = frozen_dist.cdf(x_grid)
195
196     ecdf_result = st.ecdf(sample)

```

```

195     ecdf_values = ecdf_result.cdf.evaluate(x_grid)
196     ci = ecdf_result.cdf.confidence_interval(confidence_level
197 =0.95)
198     low = ci.low.evaluate(x_grid)
199     high = ci.high.evaluate(x_grid)
200
201     ax.plot(x_grid, true_cdf, color="crimson", linewidth=2,
202 label="Истинная CDF")
203     ax.step(
204         x_grid,
205         ecdf_values,
206         where="post",
207         color="royalblue",
208         linewidth=2,
209         label="ЭФР SciPy",
210     )
211     ax.step(
212         x_grid,
213         low,
214         where="post",
215         color="forestgreen",
216         linestyle="--",
217         linewidth=1.5,
218         label="95% ДИ",
219     )
220     ax.step(
221         x_grid,
222         high,
223         where="post",
224         color="forestgreen",
225         linestyle="--",
226         linewidth=1.5,
227     )
228     ax.set_title(title)
229     ax.set_xlabel("x")
230     ax.set_ylabel("F(x)")
231     ax.set_ylim(-0.05, 1.05)
232
233 print(build_demo_table(plot_config))

```

```

233
234
235 fig, axes = plt.subplots(4, 2, figsize=(16, 18))
236 axes = axes.ravel()
237
238 for ax, item in zip(axes, plot_config):
239     sample = samples[item["sample_name"]]
240     plot_with_custom_ecdf(ax, sample, item["title"], item["
        kind"], item["dist"])
241
242 handles, labels = axes[0].get_legend_handles_labels()
243 fig.legend(handles, labels, loc="upper center", ncol=3,
        frameon=True)
244 fig.suptitle("Собственная реализация эмпирической функции рас
        пределения", fontsize=16)
245 fig.tight_layout(rect=[0, 0, 1, 0.97])
246 plt.show()
247
248
249 fig, axes = plt.subplots(4, 2, figsize=(16, 18))
250 axes = axes.ravel()
251
252 for ax, item in zip(axes, plot_config):
253     sample = samples[item["sample_name"]]
254     plot_with_scipy_ecdf(ax, sample, item["title"], item["
        kind"], item["dist"])
255
256 handles, labels = axes[0].get_legend_handles_labels()
257 fig.legend(handles, labels, loc="upper center", ncol=3,
        frameon=True)
258 fig.suptitle("Встроенная эмпирическая функция распределения и
        в scipy.stats", fontsize=16)
259 fig.tight_layout(rect=[0, 0, 1, 0.97])
260 plt.show()

```

Листинг 1: Код программы для лабораторной работы №3